

Exam Solution for TDT4186

Spring 2023

1 Multi-Choice Questions

1. **Parent:** `exec` overwrites the address space cloned from parent process, so the code after `exec` will not be executed.
2. **1, 2:** If a process is launched or preempted, it will be transitioned to *ready state*, pending for CPU resource. If a process issued an I/O request and the request is not done yet, the process is in *block state*. However, if the I/O request is done, it will be in *ready state* as well.
3. **Stack registers of threads:** Code, open files and global variables are shared by threads within a same process. Since threads usually execute different codes, they will have their own stacks which will not be shared and stack register is used to track a stack and store the current address of the stack.
4. **FIFO, RR:** In RR and FIFO, all tasks are always guaranteed to get CPU resources to execute, so no starvation.
5. **2:** Task 4 finishes first, and Task 2 finishes second. When a new task is added or a task just completes its execution, a new scheduling decision should be made for STCF. Task 1 executes for 5 and preempted by Task 2, and Task 2 executes for 5 and preempted by Task 4. At 10, Task 2 has 8 remaining, but Task 4 has only 7 execution time. Thus, Task 4 will be scheduled to execute, and then Task 2 resumes its execution.
6. **Multiple segments need only one pair of base-bound registers.:** Each segment needs a pair of base-bound registers.
7. **Segmentation Fault/Violation:** The bound register denotes the size, so 0x0400 definitely exceeds the memory range of 0x0150+0x0190 (the hexadecimal of 400). Also can do comparison in decimal format.
8. **45, (34, 26, 23, 36):** Worst-fit always selects the largest memory partition.
9. **Upon first access of a page, two memory references will be made.:** Page table is stored in memory, so the first time access always needs to access memory to find the page table in memory and then locate the page in memory.

10. **1,3:** Demanding page only loads pages when they are about to be execute. It will not load it in advance, i.e., before really requesting the pages. Every process has its own page table. Virtual memory can be larger than physical memory with swapping.
11. **0, 1, and 2:** The value cannot be -1 regardless of how they are executed. Other values can be obtained if different execution orders can be generated.
12. **Mutual Exclusion, Progress, Bounded Waiting:** Can be found from the text-book.
13. **Two semaphores are needed to implement a solution for producer-consumer problems.:** `sem_post` does not block the caller, and it only increments semaphores.
14. **Circular Queue:** The correct one should be *circular waiting*.
15. **Sector:** Sector is the concept of hard disk drive, not the on-disk structures.
16. **8:** $1GB = 2^{30}$, $4KB = 2^{12}$, so it has $2^{30-12} = 2^{18}$ data blocks. Bitmaps only need one bit for each block and $4KB$ block can track $4096 \times 8 = 2^{15}$ blocks. As a result, it needs $2^{18-15} = 8$ blocks.

2 Multi-part questions

Process

- It will have 4 processes at the end. The first `fork` creates 2, and the second `fork` will create 2 processes for each process created from the previous `fork` (including the parent process and the child process).
- `aabbbb` is the most correct one, but other solutions such as `abbabb` can also be deemed correct.

Scheduling

- 29.6. See Fig 1
- 81. See Fig 2 and Fig 3. It is easy to make a mistake when adding task E to the waiting queue. Task E actually arrives after task A completes the first time quantum, so task A will get the next turn after B and C but before E as seen in the gantt chart.
- decreased. See Fig 4. From this example, it is seen that how time slicing influences the performance.

SJF scheduling					
Job	Arrival Time	Burst Time	Finish Time	Turnaround Time	Waiting Time
A	0	17	17	17	0
B	3	21	53	50	29
C	6	32	107	101	69
D	9	15	32	23	8
E	11	22	75	64	42
Average				$255 / 5 = 51$	$148 / 5 = 29.6$

Figure 1: SJF execution table

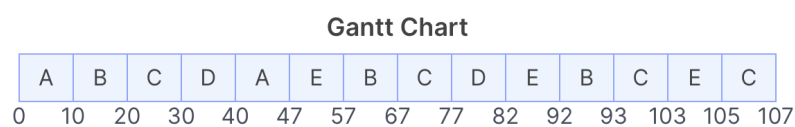


Figure 2: RR with 10 time slicing Gantt Chart

RR scheduling with quantum 10

Job	Arrival Time	Burst Time	Finish Time	Turnaround Time
A	0	17	47	47
B	3	21	93	90
C	6	32	107	101
D	9	15	82	73
E	11	22	105	94
Average				$405 / 5 = 81$

Figure 3: RR with 10 time slicing execution table

RR scheduling with time quantum 5

Job	Arrival Time	Burst Time	Finish Time	Turnaround Time
A	0	17	62	62
B	3	21	88	85
C	6	32	107	101
D	9	15	72	63
E	11	22	100	89
Average				$400 / 5 = 80$

Figure 4: RR with 5 time slicing execution table

Memory

- Fixed partition and variable partition.
- Fixed partition has internal fragmentation while variable partition has external fragmentation.

Page

- 8. Page offset is determined by page size. In this question, it is $256 = 2^8$, so it needs 8bits.
- 0x9abc. Since the least significant 8 bits are used for page offset, other bits are used for page index (VPN). The virtual address is split into two parts, where bc is offset and the first b is used to find the frame. b is 11 (decimal), and the corresponding frame is 0x9a as seen from the page table. Finally, adding offset to it leads to the correct solution.
- 2 page faults. As the question states, the page table has been loaded into TLB, so we just need to identify the pages which are not in TLB or will be accessed for the first time, i.e., the addresses that are not in the page table provided. We can use the same method shown in the previous question to determine which virtual addresses will be newly accessed. They are 0x0D17 and 0x0C15, because C and D are equal to 12 and 13, respectively.
- Two equations can be used.

$$AMAT = 2/14 * 1ms + 50ns \approx 0.14ms$$

$$AMAT = 2/14 * 1ms + (1 - 2/14) * 50ns \approx 0.14ms$$

The result does not have to be the exact 0.14ms. Alternative precision formats are also valid.

Concurrency

- 4 possible values, i.e., 4,6,7,8. The following are all possible execution orders.
A B C D 7
A C B D 4
A C D B 6
C A B D 4
C A D B 6
C D A B 8
- 2 possible values, i.e., 7, 8. Since a lock is added, only two values are possible.
A B C D 7
C D A B 8

I/O Devices

- I/O Instructions and memory-mapped I/O
- I/O instructions use special instructions to send and receive command and data to devices' registers. Memory-mapped I/O makes devices' registers available as normal memory addresses, so normal memory instructions can be used to interact with I/O devices.
- I/O instructions will not need extra memory for I/O devices. Memory-mapped I/O does not need specific instructions which may complicate the hardware design.

File System

- seek, rotation, and transfer
- 14ms (round up) or 13.67ms
average seek time is 5 ms
rotation latency = $60000 \text{ ms} / 15000 = 4\text{ms}$. Multiply by 0.5 for the average value.
transfer = $\frac{1\text{sec}}{600\text{MB}} \times 4\text{MB} = 0.007\text{s} = 7\text{ms}$ or $0.0067\text{s} = 6.67\text{ms}$.
total = $5 + 2 + 7 = 14\text{ms}$